

Аппарат сетей Петри

Задача «Обедающие философы»

Федорова Анжелика Игоревна

2026-04-18

- 1 Информация
- 2 Вводная часть
- 3 Теория
- 4 Реализация
- 5 Результаты
- 6 Literate Programming
- 7 Выводы

Раздел 1

Информация

- Федорова Анжелика Игоревна

- Федорова Анжелика Игоревна
- студентка группы НКНбд-01-23

- Федорова Анжелика Игоревна
- студентка группы НКНбд-01-23
- Российский университет дружбы народов им. П. Лумумбы

- Федорова Анжелика Игоревна
- студентка группы НКНбд-01-23
- Российский университет дружбы народов им. П. Лумумбы
- 1132236011@rudn.ru

Раздел 2

Вводная часть

- Параллельные и распределённые системы повсеместны: ОС, БД, сети

- Параллельные и распределённые системы повсеместны: ОС, БД, сети
- Проблемы синхронизации — взаимные блокировки (deadlock), голодание (starvation) — трудно отлаживать

- Параллельные и распределённые системы повсеместны: ОС, БД, сети
- Проблемы синхронизации — взаимные блокировки (deadlock), голодание (starvation) — трудно отлаживать
- Сети Петри — строгий математический аппарат для **формального** анализа таких систем

- Параллельные и распределённые системы повсеместны: ОС, БД, сети
- Проблемы синхронизации — взаимные блокировки (deadlock), голодание (starvation) — трудно отлаживать
- Сети Петри — строгий математический аппарат для **формального** анализа таких систем
- Задача «Обедающие философы» — классический учебный пример для демонстрации deadlock

- Освоить математический аппарат сетей Петри

- Освоить математический аппарат сетей Петри
- Построить сеть Петри для задачи «Обедающие философы» ($N = 5$)

- Освоить математический аппарат сетей Петри
- Построить сеть Петри для задачи «Обедающие философы» ($N = 5$)
- Обнаружить состояние **deadlock** экспериментально

- Освоить математический аппарат сетей Петри
- Построить сеть Петри для задачи «Обедающие философы» ($N = 5$)
- Обнаружить состояние **deadlock** экспериментально
- Модифицировать сеть с **арбитром** для предотвращения тупика

- Освоить математический аппарат сетей Петри
- Построить сеть Петри для задачи «Обедающие философы» ($N = 5$)
- Обнаружить состояние **deadlock** экспериментально
- Модифицировать сеть с **арбитром** для предотвращения тупика
- Провести стохастическое и детерминированное моделирование

- Освоить математический аппарат сетей Петри
- Построить сеть Петри для задачи «Обедающие философы» ($N = 5$)
- Обнаружить состояние **deadlock** экспериментально
- Модифицировать сеть с **арбитром** для предотвращения тупика
- Провести стохастическое и детерминированное моделирование
- Визуализировать результаты: графики маркировки, анимация GIF

- Язык программирования **Julia**

- Язык программирования **Julia**
- Пакеты: `OrdinaryDiffEq`, `Plots`, `DataFrames`, `CSV`, `Literate`

- Язык программирования **Julia**
- Пакеты: `OrdinaryDiffEq`, `Plots`, `DataFrames`, `CSV`, `Literate`
- Управление проектом: **DrWatson**

- Язык программирования **Julia**
- Пакеты: `OrdinaryDiffEq`, `Plots`, `DataFrames`, `CSV`, `Literate`
- Управление проектом: **DrWatson**
- Стохастическое моделирование: **алгоритм Гиллеспи**

- Язык программирования **Julia**
- Пакеты: OrdinaryDiffEq, Plots, DataFrames, CSV, Literate
- Управление проектом: **DrWatson**
- Стохастическое моделирование: **алгоритм Гиллеспи**
- Детерминированное моделирование: система ОДУ (метод Tsit5)

- Язык программирования **Julia**
- Пакеты: OrdinaryDiffEq, Plots, DataFrames, CSV, Literate
- Управление проектом: **DrWatson**
- Стохастическое моделирование: **алгоритм Гиллеспи**
- Детерминированное моделирование: система ОДУ (метод Tsit5)
- Literate Programming: генерация скрипта, Jupyter Notebook, Quarto

Раздел 3

Теория

Сеть Петри — это четвёрка:

$$N = (P, T, F, M_0)$$

- $P = \{p_1, p_2, \dots, p_n\}$ — конечное множество **позиций**

Сеть Петри — это четвёрка:

$$N = (P, T, F, M_0)$$

- $P = \{p_1, p_2, \dots, p_n\}$ — конечное множество **позиций**
- $T = \{t_1, t_2, \dots, t_m\}$ — конечное множество **переходов**

Сеть Петри — это четвёрка:

$$N = (P, T, F, M_0)$$

- $P = \{p_1, p_2, \dots, p_n\}$ — конечное множество **позиций**
- $T = \{t_1, t_2, \dots, t_m\}$ — конечное множество **переходов**
- $F \subseteq (P \times T) \cup (T \times P)$ — множество **дуг**

Сеть Петри — это четвёрка:

$$N = (P, T, F, M_0)$$

- $P = \{p_1, p_2, \dots, p_n\}$ — конечное множество **позиций**
- $T = \{t_1, t_2, \dots, t_m\}$ — конечное множество **переходов**
- $F \subseteq (P \times T) \cup (T \times P)$ — множество **дуг**
- $M_0 : P \rightarrow \mathbb{N}$ — начальная **маркировка**

Позиции (кружки)

- Пассивные элементы

Переходы (прямоугольники)

Правило срабатывания:

Переход **разрешён**, если каждая входная позиция содержит достаточно фишек. При срабатывании:

Позиции (кружки)

- Пассивные элементы
- Описывают состояние системы

Переходы (прямоугольники)

Правило срабатывания:

Переход **разрешён**, если каждая входная позиция содержит достаточно фишек. При срабатывании:

Позиции (кружки)

- Пассивные элементы
- Описывают состояние системы
- Содержат **фишки** — количество ресурсов

Переходы (прямоугольники)

Правило срабатывания:

Переход **разрешён**, если каждая входная позиция содержит достаточно фишек. При срабатывании:

Позиции (кружки)

- Пассивные элементы
- Описывают состояние системы
- Содержат **фишки** — количество ресурсов

Переходы (прямоугольники)

- Активные элементы

Правило срабатывания:

Переход **разрешён**, если каждая входная позиция содержит достаточно фишек. При срабатывании:

Позиции (кружки)

- Пассивные элементы
- Описывают состояние системы
- Содержат **фишки** — количество ресурсов

Переходы (прямоугольники)

- Активные элементы
- Моделируют события и действия

Правило срабатывания:

Переход **разрешён**, если каждая входная позиция содержит достаточно фишек. При срабатывании:

Позиции (кружки)

- Пассивные элементы
- Описывают состояние системы
- Содержат **фишки** — количество ресурсов

Переходы (прямоугольники)

- Активные элементы
- Моделируют события и действия

Правило срабатывания:

Переход **разрешён**, если каждая входная позиция содержит достаточно фишек. При срабатывании:

- фишки из входных позиций **удаляются**

Позиции (кружки)

- Пассивные элементы
- Описывают состояние системы
- Содержат **фишки** — количество ресурсов

Переходы (прямоугольники)

- Активные элементы
- Моделируют события и действия

Правило срабатывания:

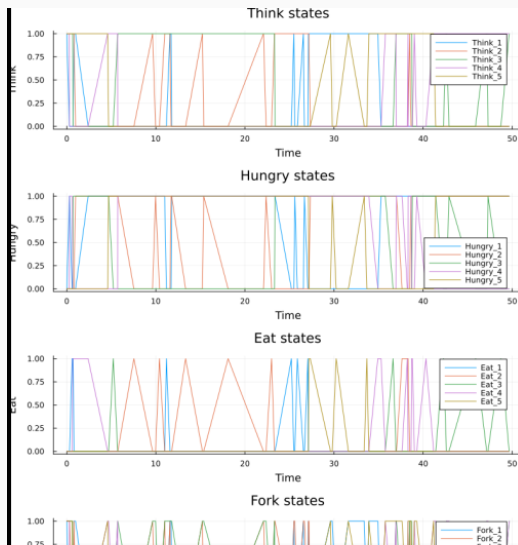
Переход **разрешён**, если каждая входная позиция содержит достаточно фишек. При срабатывании:

- фишки из входных позиций **удаляются**
- фишки в выходных позициях **добавляются**

Задача «Обедающие философы»

- $N = 5$ философов сидят за круглым столом

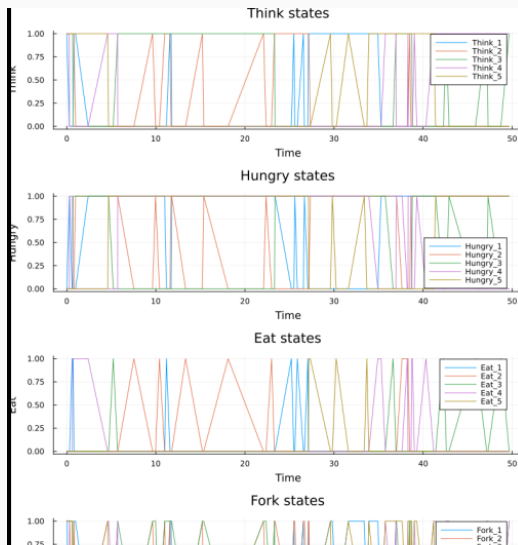
Проблема: при наивной реализации все философы берут левую вилку и **ждут правую** → система замирает



Задача «Обедающие философы»

- $N = 5$ философов сидят за круглым столом
- Между каждыми двумя соседями — одна **вилка**

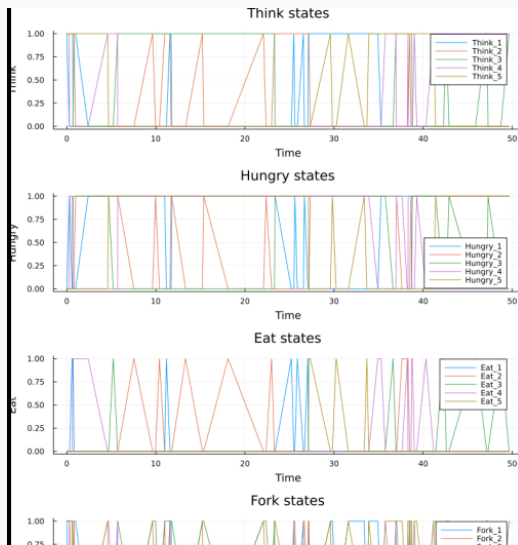
Проблема: при наивной реализации все философы берут левую вилку и **ждут правую** → система замирает



Задача «Обедающие философы»

- $N = 5$ философов сидят за круглым столом
- Между каждыми двумя соседями — одна **вилка**
- Чтобы есть — нужны **обе вилки** (левая и правая)

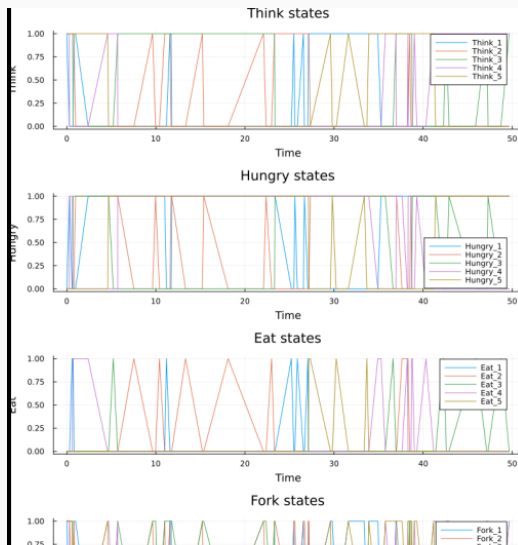
Проблема: при наивной реализации все философы берут левую вилку и **ждут правую** → система замирает



Задача «Обедающие философы»

- $N = 5$ философов сидят за круглым столом
- Между каждыми двумя соседями — одна **вилка**
- Чтобы есть — нужны **обе вилки** (левая и правая)
- Три состояния: **думает** → **голоден** → **ест**

Проблема: при наивной реализации все философы берут левую вилку и **ждут правую** → система замирает



Раздел 4

Реализация

Структура модуля DiningPhilosophers.jl

```
struct PetriNet
    n_places::Int
    n_transitions::Int
    incidence::Matrix{Int}    # матрица инцидентности
    place_names::Vector{Symbol}
    transition_names::Vector{Symbol}
end
```

- **4N позиций:** Think_i, Hungry_i, Eat_i, Fork_i

Структура модуля DiningPhilosophers.jl

```
struct PetriNet
    n_places::Int
    n_transitions::Int
    incidence::Matrix{Int}    # матрица инцидентности
    place_names::Vector{Symbol}
    transition_names::Vector{Symbol}
end
```

- **4N позиций:** Think_i, Hungry_i, Eat_i, Fork_i
- **3N переходов:** GetLeft_i, GetRight_i, PutForks_i

Структура модуля DiningPhilosophers.jl

```
struct PetriNet
    n_places::Int
    n_transitions::Int
    incidence::Matrix{Int}    # матрица инцидентности
    place_names::Vector{Symbol}
    transition_names::Vector{Symbol}
end
```

- **4N позиций:** Think_i, Hungry_i, Eat_i, Fork_i
- **3N переходов:** GetLeft_i, GetRight_i, PutForks_i
- Начальная маркировка: Think_i = 1, Fork_i = 1

Классическая сеть: дуги переходов

Философ i берёт левую вилку: Think $\rightarrow$ Hungry

```
add_arc!(net, think,      get_left, -1)
```

```
add_arc!(net, left_fork,  get_left, -1)
```

```
add_arc!(net, hungry,     get_left, +1)
```

Берёт правую вилку: Hungry $\rightarrow$ Eat

```
add_arc!(net, hungry,     get_right, -1)
```

```
add_arc!(net, right_fork, get_right, -1)
```

```
add_arc!(net, eat,        get_right, +1)
```

Кладёт вилки: Eat $\rightarrow$ Think

```
add_arc!(net, eat,        put_forks, -1)
```

```
add_arc!(net, think,      put_forks, +1)
```

Алгоритм Гиллеспи (стохастика)

```
while t < tmax
    # 1. Вычислить пропенсити всех переходов
    a[j] = rate * prod(u[i]^(-incidence[i,j])) for i where incidence<0
    a0 = sum(a)
    if a0 == 0; break; end    # deadlock!

    # 2. Случайный шаг времени
    dt = -log(rand()) / a0

    # 3. Выбрать переход пропорционально пропенсити
    chosen = sample(transitions, weights=a)

    # 4. Обновить маркировку
```

Инициализация проекта

```
julia
```

```
using DrWatson
```

```
initialize_project("project"; authors=["Fyodorova Angelica"])
```

```
julia> initialize_project("project"; authors=["Fyodorova Angelica"])
  Activating new project at `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project`
    Updating registry at `~/.julia/registries/General.toml`
    Resolving package versions...
    Updating `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project/Project.toml`
 [634d3b9d] + DrWatson v2.19.1
    Updating `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project/Manifest.toml`
 [0b6fb165] + ChunkCodecCore v1.0.1
 [4c0bbe4] + ChunkCodecLibZlib v1.0.0
```

Раздел 5

Результаты

Запуск моделирования

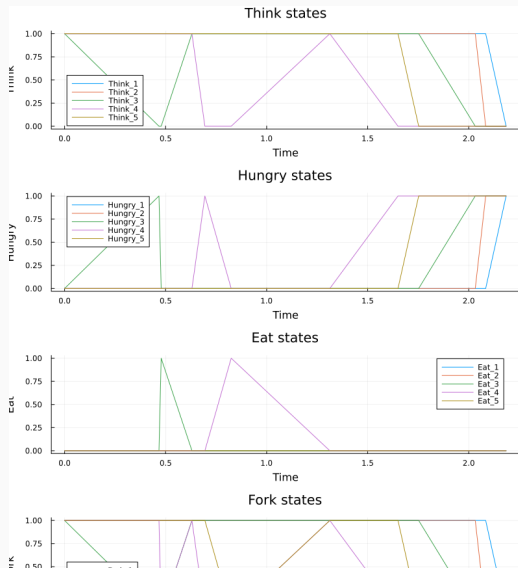
```
N = 5;  tmax = 50.0
```

```
net_classic, u0, _ = build_classical_network(N)
df = simulate_stochastic(net_classic, u0, tmax)
dead = detect_deadlock(df, net_classic)
println("Deadlock обнаружен: $dead")
```

```
ed.
1 dependency successfully precompiled in 2 seconds. 42 already precompiled.
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true

=== Сеть с арбитром ===
Could not create decoration from factory! Running with no decorations.
Deadlock обнаружен: false
ai.fedorova@ai.fedorova: ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--stu
```

Классическая сеть: deadlock



- До $t \approx 2.2$: философы успевают поест по 1-2 раза

- До $t \approx 2.2$: философы успевают поест по 1-2 раза
- Затем **все** переходят в состояние Hungry

- До $t \approx 2.2$: философы успевают поесть по 1-2 раза
- Затем **все** переходят в состояние Hungry
- $Eat_i = 0$ для всех i — никто не ест

- До $t \approx 2.2$: философы успевают поесть по 1-2 раза
- Затем **все** переходят в состояние Hungry
- $Eat_i = 0$ для всех i — никто не ест
- $Fork_i = 0$ для всех i — все вилки «захвачены»

- До $t \approx 2.2$: философы успевают поест по 1-2 раза
- Затем **все** переходят в состояние Hungry
- $Eat_i = 0$ для всех i — никто не ест
- $Fork_i = 0$ для всех i — все вилки «захвачены»
- `detect_deadlock` возвращает **true**

- До $t \approx 2.2$: философы успевают поест по 1-2 раза
- Затем **все** переходят в состояние Hungry
- $Eat_i = 0$ для всех i — никто не ест
- $Fork_i = 0$ для всех i — все вилки «захвачены»
- `detect_deadlock` возвращает **true**
- Система **заморожена** навсегда

Добавляется позиция **Arbiter** с $N - 1 = 4$ фишками.

Переход `GetLeft_i` требует **фишку арбитра**:

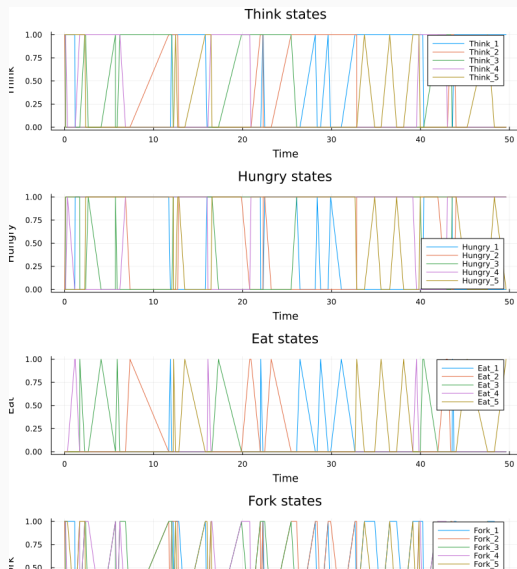
```
add_arc!(net, arbiter_idx, get_left, -1)
```

Переход `PutForks_i` возвращает фишку:

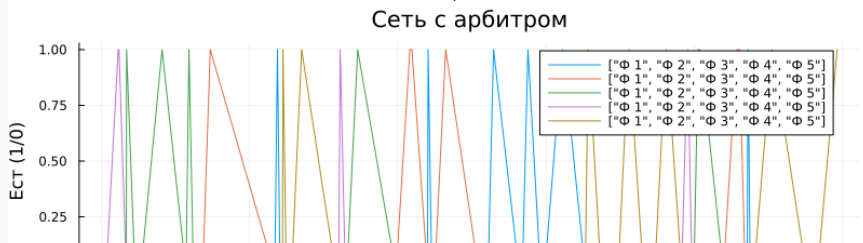
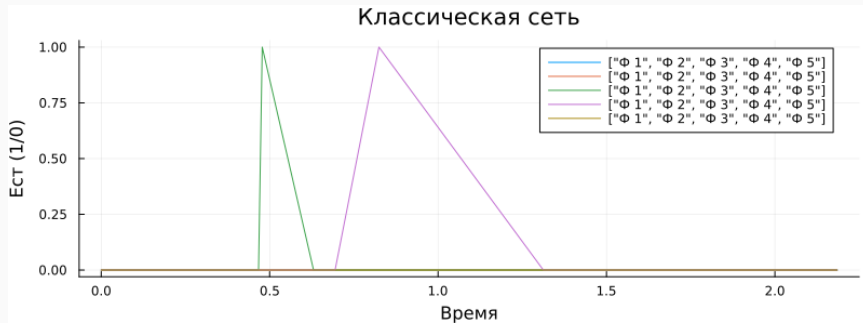
```
add_arc!(net, arbiter_idx, put_forks, +1)
```

Эффект: не более 4 философов
одновременно могут взять левую вилку
→ круговое ожидание невозможно

Сеть с арбитром: результат



Сравнительный анализ



Классическая сеть

- Активность Eat_i прекращается к $t \approx 1.5$

Сеть с арбитром

Классическая сеть

- Активность Eat_i прекращается к $t \approx 1.5$
- Система необратимо блокируется

Сеть с арбитром

Классическая сеть

- Активность Eat_i прекращается к $t \approx 1.5$
- Система необратимо блокируется
- $detect_deadlock \rightarrow \mathbf{true}$

Сеть с арбитром

Классическая сеть

- Активность Eat_i прекращается к $t \approx 1.5$
- Система необратимо блокируется
- `detect_deadlock` → **true**

Сеть с арбитром

- Философы непрерывно чередуются

Классическая сеть

- Активность Eat_i прекращается к $t \approx 1.5$
- Система необратимо блокируется
- $detect_deadlock \rightarrow \mathbf{true}$

Сеть с арбитром

- Философы непрерывно чередуются
- Нет длительных нулевых периодов

Классическая сеть

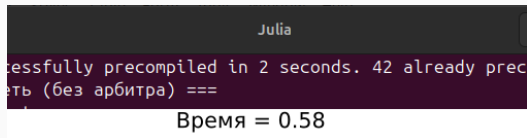
- Активность Eat_i прекращается к $t \approx 1.5$
- Система необратимо блокируется
- $detect_deadlock \rightarrow \mathbf{true}$

Сеть с арбитром

- Философы непрерывно чередуются
- Нет длительных нулевых периодов
- $detect_deadlock \rightarrow \mathbf{false}$

Анимация сети Петри

```
anim = @animate for row in eachrow(df)
    u = [row[col] for col in propertynames(row)
          if col != :time]
    bar(1:length(u), u,
        title = "Время = $(round(row.time, digits=2))",
        xlabel = "Позиция", ylabel = "Фишки")
    xticks!(1:length(u), string.(names), rotation=45)
end
gif(anim, plotsdir("philosophers_simulation.gif"), fps=2)
```



Раздел 6

Literate Programming

Генерация артефактов из одного источника

using Literate

Чистый скрипт Julia

```
Literate.script("src/DiningPhilosophers.jl", "scripts/")
```

Jupyter Notebook (с выполнением)

```
Literate.notebook("src/DiningPhilosophers.jl", "notebooks/")
```

Документация Markdown / Quarto

```
Literate.markdown("src/DiningPhilosophers.jl", "docs/")
```

```
julia> Literate.script("src/DiningPhilosophers.jl", "scripts/")  
[ Info: generating plain script file from `~/work/study/2026-1/2026-1==study--simulation  
-modeling/2026-1--study--simulation-modeling/labs/lab05/project/src/DiningPhilosophers.j
```

- `scripts/DiningPhilosophers.jl` — чистый исполняемый скрипт

- `scripts/DiningPhilosophers.jl` — чистый исполняемый скрипт
- `notebooks/DiningPhilosophers.ipynb` — Jupyter Notebook, выполнен автоматически

- `scripts/DiningPhilosophers.jl` — чистый исполняемый скрипт
- `notebooks/DiningPhilosophers.ipynb` — Jupyter Notebook, выполнен автоматически
- `docs/DiningPhilosophers.md` — документация для Quarto

- `scripts/DiningPhilosophers.jl` — чистый исполняемый скрипт
- `notebooks/DiningPhilosophers.ipynb` — Jupyter Notebook, выполнен автоматически
- `docs/DiningPhilosophers.md` — документация для Quarto
- Единый источник правды — изменение кода в `.jl` обновляет все артефакты

Раздел 7

Выводы

- ❶ **Построена** сеть Петри для $N = 5$ философов: 20 позиций, 15 переходов

- ❶ **Построена** сеть Петри для $N = 5$ философов: 20 позиций, 15 переходов
- ❷ **Deadlock подтверждён** экспериментально — стохастическая симуляция неизбежно приходит к тупику

- ❶ **Построена** сеть Петри для $N = 5$ философов: 20 позиций, 15 переходов
- ❷ **Deadlock подтверждён** экспериментально — стохастическая симуляция неизбежно приходит к тупику
- ❸ **Арбитр решает проблему** — 4 фишки в позиции Arbiter исключают круговое ожидание

- ❶ **Построена** сеть Петри для $N = 5$ философов: 20 позиций, 15 переходов
- ❷ **Deadlock подтверждён** экспериментально — стохастическая симуляция неизбежно приходит к тупику
- ❸ **Арбитр решает проблему** — 4 фишки в позиции Arbiter исключают круговое ожидание
- ❹ **Визуализация** (графики, GIF-анимация, итоговый отчёт) наглядно демонстрирует разницу между моделями

- ➊ **Построена** сеть Петри для $N = 5$ философов: 20 позиций, 15 переходов
- ➋ **Deadlock подтверждён** экспериментально — стохастическая симуляция неизбежно приходит к тупику
- ➌ **Арбитр решает проблему** — 4 фишки в позиции Arbiter исключают круговое ожидание
- ➍ **Визуализация** (графики, GIF-анимация, итоговый отчёт) наглядно демонстрирует разницу между моделями
- ➎ **Инструменты** DrWatson + Literate.jl обеспечивают воспроизводимость и структуру проекта

Принципы, изученные на задаче философов, применимы к реальным системам:

- **Базы данных** — конкурентный доступ к таблицам, deadlock транзакций

Сеть Петри с арбитром — аналог **семафора** в операционных системах.

Принципы, изученные на задаче философов, применимы к реальным системам:

- **Базы данных** — конкурентный доступ к таблицам, deadlock транзакций
- **Операционные системы** — планирование процессов, управление ресурсами

Сеть Петри с арбитром — аналог **семафора** в операционных системах.

Принципы, изученные на задаче философов, применимы к реальным системам:

- **Базы данных** — конкурентный доступ к таблицам, deadlock транзакций
- **Операционные системы** — планирование процессов, управление ресурсами
- **Сетевые протоколы** — синхронизация распределённых узлов

Сеть Петри с арбитром — аналог **семафора** в операционных системах.

Принципы, изученные на задаче философов, применимы к реальным системам:

- **Базы данных** — конкурентный доступ к таблицам, deadlock транзакций
- **Операционные системы** — планирование процессов, управление ресурсами
- **Сетевые протоколы** — синхронизация распределённых узлов
- **Производство** — моделирование гибких производственных линий

Сеть Петри с арбитром — аналог **семафора** в операционных системах.

- Методическое пособие по дисциплине «Имитационное моделирование», лабораторная работа №5: Аппарат сетей Петри.

- Методическое пособие по дисциплине «Имитационное моделирование», лабораторная работа №5: Аппарат сетей Петри.
- Petri C. A. «Kommunikation mit Automaten». Dissertation, Universität Bonn, 1962.

- Методическое пособие по дисциплине «Имитационное моделирование», лабораторная работа №5: Аппарат сетей Петри.
- Petri C. A. «Kommunikation mit Automaten». Dissertation, Universität Bonn, 1962.
- Dijkstra E. W. «Cooperating Sequential Processes». Technical Report EWD-123, 1965.